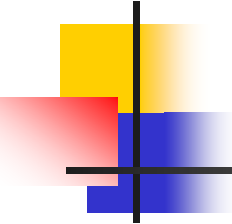


算法设计与分析

Analysis and Design of Algorithm

Lesson 04



要点回顾

- 第一章**结束**

- NP完全性理论 (P、NP、NPC、NP-Hard)

- 第二章**开端**

- 递归的概念

- 递归的例子

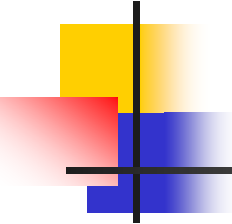
- 阶乘、Fibonacci数列、双递归函数

双递归函数 Ackerman函数

- 一个函数与它的一个变量是由函数自身定义。

$$A(n, m): \begin{cases} A(1, 0) = 2 & \text{①} \\ A(0, m) = 1 & m \geq 0 \quad \text{②} \\ A(n, 0) = n + 2 & n \geq 2 \quad \text{③} \\ A(n, m) = A(A(n-1, m), m-1) & n, m \geq 1 \quad \text{④} \end{cases}$$

1. $m=0, A(n, 0)=n+2$
2. $m=1, A(n, 1)=2n$



要点回顾

- 第一章**结束**

- NP完全性理论 (P、NP、NPC、NP-Hard)

- 第二章**开端**

- 递归的概念

- 递归的例子

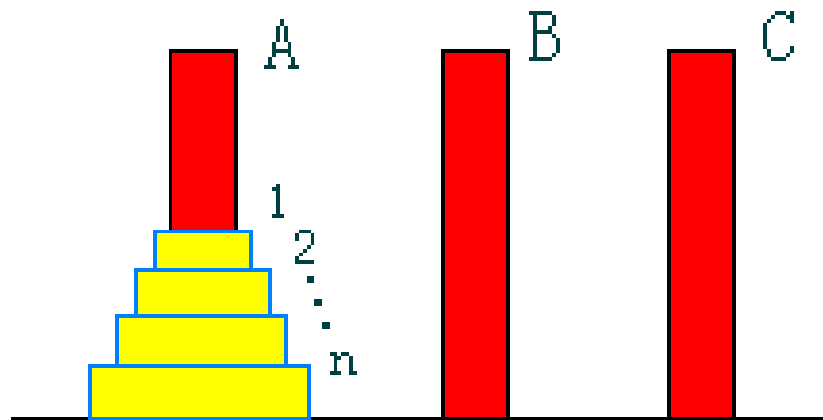
- 阶乘、Fibonacci数列、双递归函数
 - 正整数划分

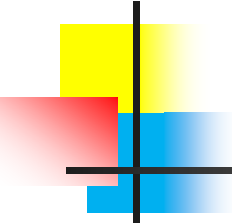


Hanoi塔传说

Hanoi塔问题

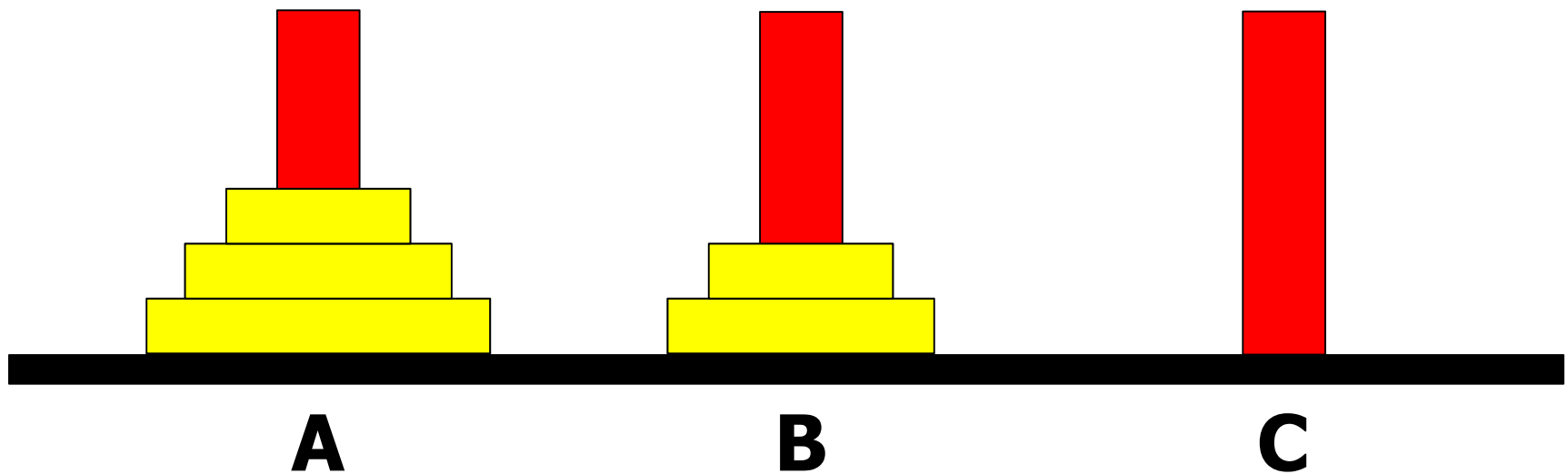
- 设A, B, C是3个塔座。开始时，在塔座A上有一叠共 n 个圆盘，这些圆盘自下而上，由大到小地叠在一起。各圆盘从小到大编号为 $1, 2, \dots, n$ ，现要求将塔座A上的这一叠圆盘移到塔座C上，并仍按同样顺序叠置。在移动圆盘时应遵守以下移动规则：
 - 规则1：每次只能移动1个圆盘；
 - 规则2：任何时刻都不允许将较大的圆盘压在较小的圆盘之上；
 - 规则3：满足规则1和2的前提下，可将圆盘移至A, B, C任一塔座上。





Hanoi塔问题

- 先来看一个简单的实例



递归算法

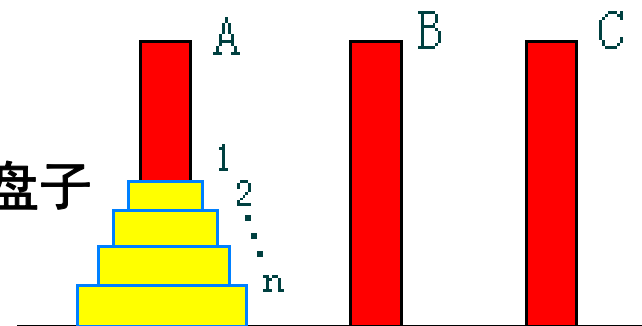
■ 算法Hanoi(n , A, B, C) // n 个盘子借助B, 从A移到C

1. **if** $n = 1$ **then** move (A, C)

2. **else** Hanoi($n-1$, A, C, B)

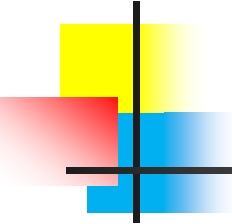
3. move (A, C) //剩下的一个盘子

4. Hanoi($n-1$, B, A, C)



■ **优点：**简单、优雅、易于理解；

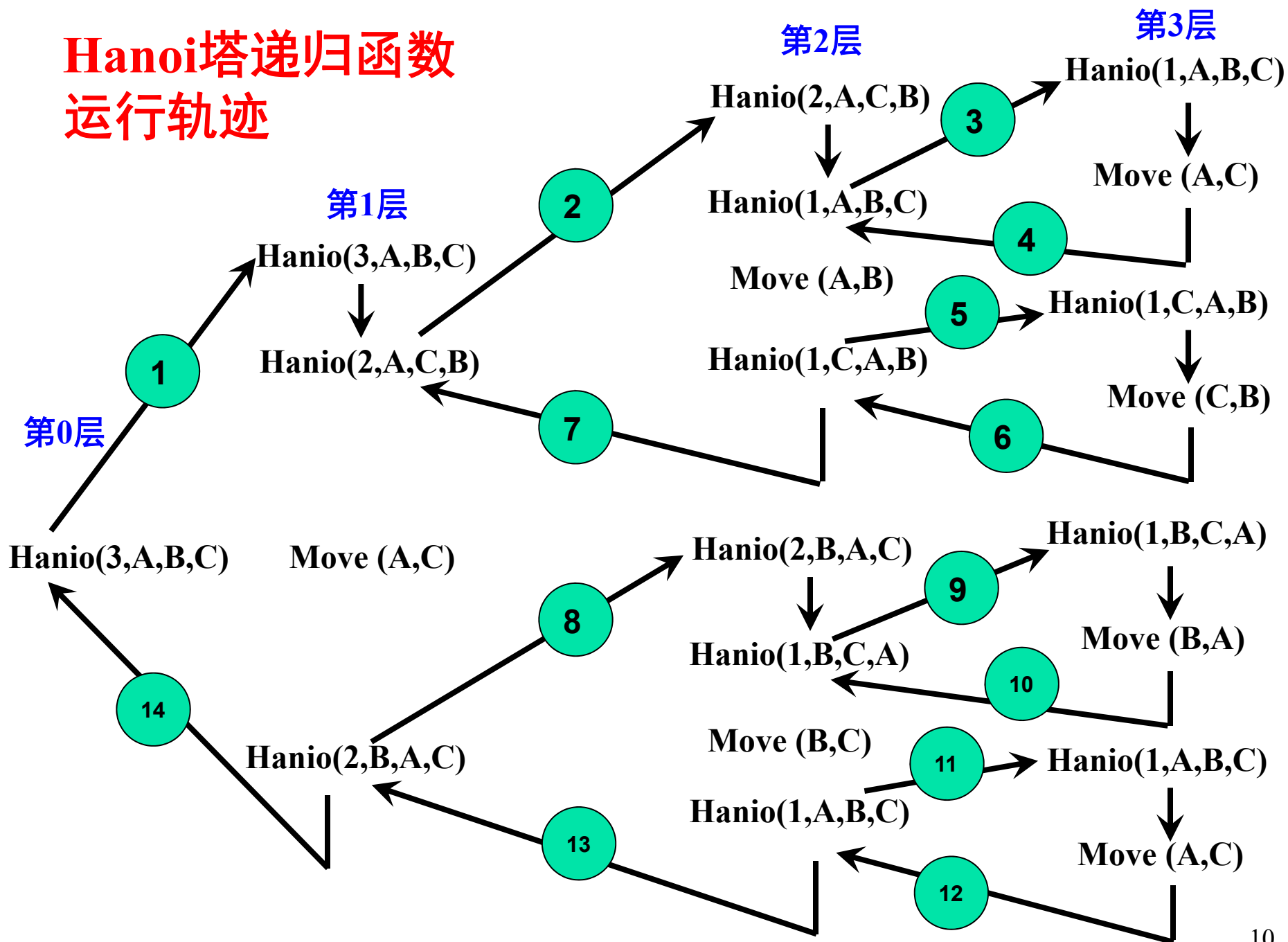
■ **缺点：**算法Hanoi以递归形式给出，每个圆盘的具体移动方式并不清楚，因此当 $n \geq 5$ 以后，很难用手工移动来模拟这个算法。



递归函数的运行轨迹

- 在递归函数中，调用函数和被调用函数是同一个函数，需要注意的是递归函数的调用层次，如果把调用递归函数的主函数称为**第0层**，进入函数后，首次递归调用自身称为**第1层**调用；从第 i 层递归调用自身称为**第 $i+1$ 层**。反之，退出第 $i+1$ 层调用应该返回第 i 层。
- 采用**图示方法**描述递归函数的**运行轨迹**，从中可较直观地了解到各调用层次及其执行情况。

Hanoi塔递归函数 运行轨迹



递归算法

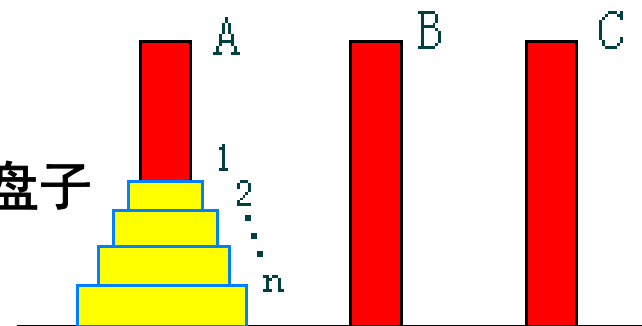
■ 算法Hanoi(n , A, B, C) // n 个盘子借助B, 从A移到C

1. **if** $n = 1$ **then** move (A, C)

2. **else** Hanoi($n-1$, A, C, B)

3. move (A, C) //剩下的一个盘子

4. Hanoi($n-1$, B, A, C)



如果用了递归，就找不到while/for循环语句。
此时，**该如何计算时间复杂度呢？**

递归算法

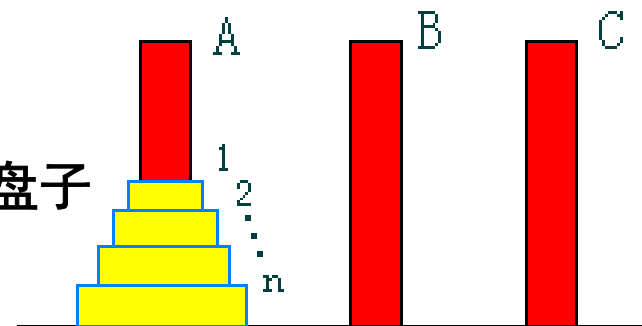
■ 算法 $\text{Hanoi}(n, A, B, C)$ // n 个盘子借助 B , 从 A 移到 C

1. if $n = 1$ then move (A, C)

2. else $\text{Hanoi}(n-1, A, C, B)$

3. move (A, C) // 剩下的一个盘子

4. $\text{Hanoi}(n-1, B, A, C)$

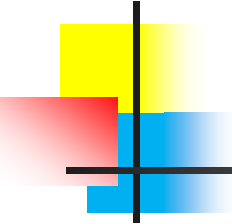


设 n 个盘子的移动次数为 $T(n)$

$$T(n) = 2T(n-1) + 1$$

$$T(1) = 1$$

➡ 递推方程



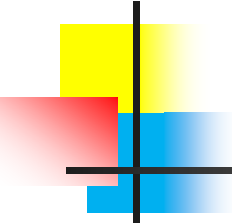
递推方程

- 设序列 $a_0, a_1, \dots, a_n, \dots$ 简记为 $\{a_n\}$ ，一个把 a_n 与某些个 $a_i (i < n)$ 联系起来的等式叫做关于序列 $\{a_n\}$ 的递推方程
- 递推方程的求解：
给定关于序列 $\{a_n\}$ 的递推方程和若干初值，
计算 a_n



迭代法求解递推方程

- 不断用递推方程的右部替换左部
- 每次替换，随着 n 的降低在和式中多出一项
- 直到出现初值停止迭代
- 将初值代入并对和式求和
- 可用数学归纳法验证解的正确性



Hanoi塔算法

$$T(n)=2T(n-1)+1$$

$$=2[2T(n-2)+1]+1$$

$$=2^2T(n-2)+2+1$$

$$= \dots\dots$$

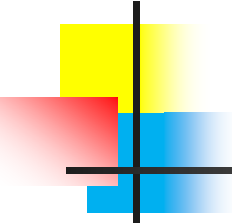
$$=2^{n-1}T(1)+2^{n-2}+2^{n-3}+\dots+2+1$$

$$=2^{n-1}+2^{n-1}-1$$

$$=2^n-1$$

$$T(n)=2T(n-1)+1$$

$$T(1)=1$$



解的正确性-归纳验证

- **证明：** 下述递推方程的解是 $T(n)=2^n-1$

- $T(n)=2T(n-1)+1$

- $T(1)=1$

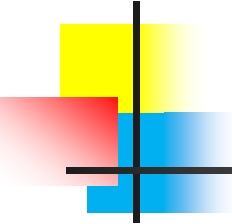
- **方法：** 数学归纳法

- **证** $n=1, T(1)=2^1-1=1$

假设对于 n ，解满足方程，则

$$T(n+1)$$

$$=2T(n)+1=2(2^n-1)+1=2^{n+1}-1$$



Hanoi塔算法

$$T(n)=2T(n-1)+1$$

$$=2[2T(n-2)+1]+1$$

$$=2^2T(n-2)+2+1$$

$$= \dots\dots$$

$$=2^{n-1}T(1)+2^{n-2}+2^{n-3}+\dots+2+1$$

$$=2^{n-1}+2^{n-1}-1$$

$$=2^n-1$$

$$T(n)=2T(n-1)+1$$

$$T(1)=1$$

5845亿年!!!

问题： 如果1秒移动1个，64个金蝶要多少时间？



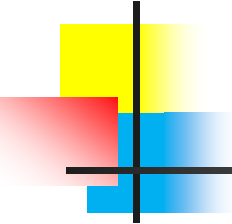
每秒能算60万亿次
要计算三天左右

Hanoi塔算法

有没有更好的算法???

没有!!!

这是一个难解的问题，不存在多项式时间的算法！



递归小结(cont.)

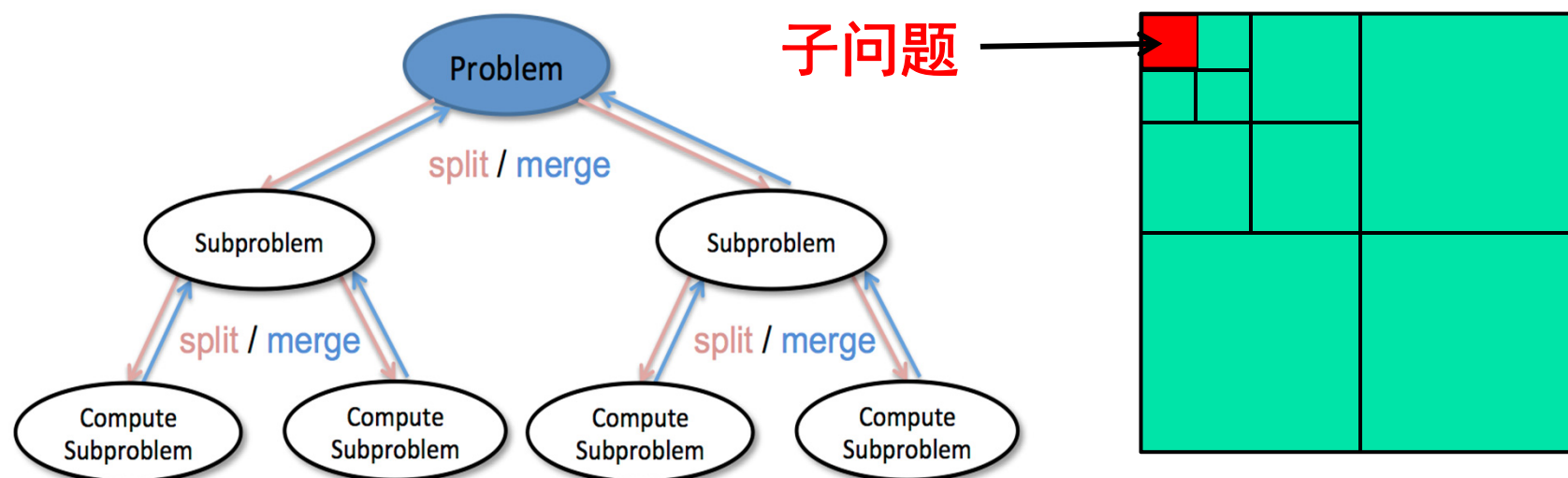
优点：结构清晰，可读性强，而且容易用数学归纳法来证明算法的正确性，因此它为设计算法、调试程序带来很大方便。

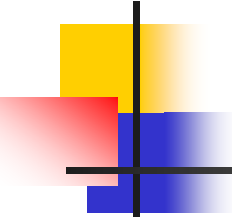
缺点：递归算法的运行效率较低，无论是耗费的计算时间还是占用的存储空间都比非递归算法要多。

分治策略

分治法(Divide-and-Conquer)

基本思想： 将一个规模为 n 的问题分解为 k 个规模较小的子问题，这些子问题**互相独立**且与原问题**相同**。递归解这些子问题，再将子问题合并得到原问题的解。



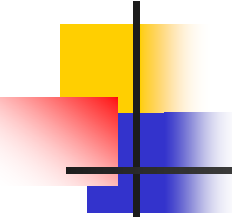


分治法的适用条件

分治法所能解决的问题一般具有以下几个特征：

- 该问题的规模缩小到一定的程度就可以容易地解决；

因为问题的计算复杂性一般是随着问题规模的增加而增加，因此大部分问题满足这个特征。

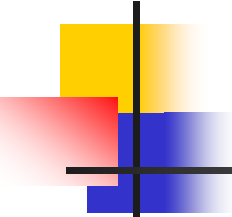


分治法的适用条件

分治法所能解决的问题一般具有以下几个特征：

- 该问题的规模缩小到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的相同问题，且该问题具有**最优子结构性质**；

这条特征是应用分治法的前提，它也是大多数问题可以满足的，此特征反映了递归思想的应用。

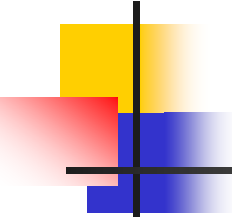


分治法的适用条件

分治法所能解决的问题一般具有以下几个特征：

- 该问题的规模缩小到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的相同问题，即该问题具有**最优子结构性质**；
- 利用该问题分解出的子问题的解可以合并为该问题的解；

能否利用分治法完全取决于问题是否具有这条特征，如果具备了前两条特征，而不具备第三条特征，则可以考虑**贪心算法**或**动态规划**。

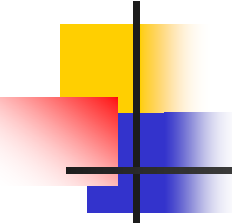


分治法的适用条件

分治法所能解决的问题一般具有以下几个特征：

- 该问题的规模缩小到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的相同问题，即该问题具有**最优子结构性质**；
- 利用该问题分解出的子问题的解可以合并为该问题的解；
- 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

该特征涉及到分治法效率，如果各子问题不独立，则分治法要做许多不必要的工作，重复地解公共子问题，此时虽也可用分治法，但一般用**动态规划**较好。

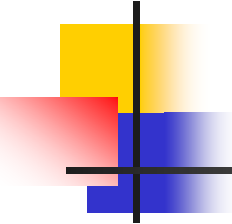


分治法的基本步骤

伪代码：

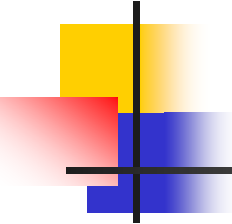
```
divide-and-conquer (P) {  
    if ( $|P| \leq n_0$ ) adhoc (P);    // (1) 解决小规模的问题  
    divide P into smaller sub instances  $P_1, P_2, \dots, P_k$ ;  
    // (2) 分解问题  
    for ( $i=1, i \leq k, i++$ )  
         $y_i = \text{divide-and-conquer}(P_i)$ ;    // (3) 递归地解各子问题  
    return merge ( $y_1, \dots, y_k$ ); // (4) 将各子问题的解合并为原问题的解  
}
```

- 其中， $|P|$ 是问题P的规模， n_0 为一阈值，表示当问题P的规模不超过 n_0 时，问题已容易解决，不必再继续分解。
- **adhoc**(P)是分治的基本子算法，用于直接解小规模的问题P。
- **merge**($y_1, y_2, \dots, y_k$)是分治算法中的合并子算法。



分治法的问题

- 将问题分为多少个子问题？
- 子问题的规模怎样才恰当？
- 研究者从大量实践中发现，在用分治法设计算法时，最好使子问题的规模大致相同。即，将一个问题的分成**大小相等**的 k 个子问题的处理方法是行之有效的。
- 这种使子问题规模大致相等的做法是出自一种叫做**平衡(balancing)子问题**的思想，它几乎总是比子问题规模不等的做法要好。



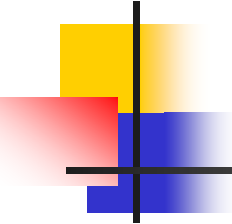
分治法的计算效率

伪代码：

```
divide-and-conquer (P) {  
    if ( $|P| \leq n_0$ ) adhoc (P);    // (1) 解决小规模的问题  
    divide P into smaller sub instances  $P_1, P_2, \dots, P_k$ ;  
    // (2) 分解问题  
    for ( $i=1, i \leq k, i++$ )  
         $y_i = \text{divide-and-conquer}(P_i)$ ;    // (3) 递归地解各子问题  
    return merge ( $y_1, \dots, y_k$ ); // (4) 将各子问题的解合并为原问题的解  
}
```

假定：

- 分成 k 个规模为 n/m 的子问题去解。
- 解规模为1的问题耗费1个单位时间。
- 分解为 k 个子问题和将 k 个子问题的解合并需用 $f(n)$ 个单位时间。



分治法的计算效率

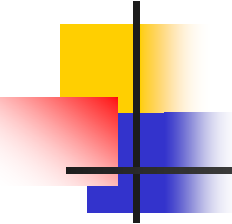
伪代码：

```
divide-and-conquer (P) {  
    if (|P| ≤ n0) adhoc (P);    // (1) 解决小规模的问题  
    divide P into smaller sub instances P1, P2, ..., Pk;  
    // (2) 分解问题  
    for (i=1, i≤k, i++)  
        Yi=divide-and-conquer (Pi);    // (3) 递归地解各子问题  
    return merge (Y1, ..., Yk); // (4) 将各子问题的解合并为原问题的解  
}
```

→ 用 $T(n)$ 表示该分治法解规模为 $|P|=n$ 的问题所需的计算时间：

$$T(n) = \begin{cases} O(1) & n = 1 \\ kT(n/m) + f(n) & n > 1 \end{cases}$$

迭代法求解方程解： $T(n)=?$



分治法的计算效率

$$T(n) = \begin{cases} O(1) & n = 1 \\ kT(n/m) + f(n) & n > 1 \end{cases}$$

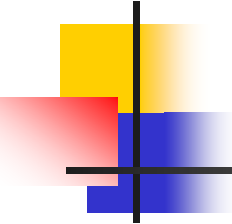
迭代法求解方程：

$$T(n) = n^{\log_m k} + \sum_{j=0}^{\log_m n - 1} k^j f(n / m^j)$$

分治法实例

从一个游戏开始😊





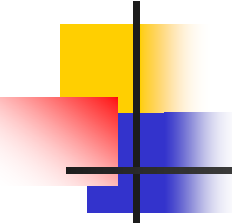
二分搜索技术

问题： 给定已按升序排好序的 n 个元素 $a[0:n-1]$ ，现要在这 n 个元素中找出一特定元素 x 。

分析：

1. 该问题的规模缩小到一定的程度就可以容易地解决；

如果 $n=1$ 即只有一个元素，则只要比较这个元素和 x 就可以确定 x 是否在表中。因此这个问题满足分治法的第一个适用条件



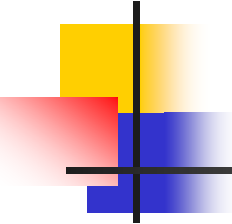
二分搜索技术

问题： 给定已按升序排好序的 n 个元素 $a[0:n-1]$ ，现在要在这 n 个元素中找出一特定元素 x 。

分析：

1. 该问题的规模缩小到一定的程度就可以容易地解决；
2. 该问题可以分解为若干个规模较小的相同问题；
3. 分解出的子问题的解可以合并为原问题的解；

比较 x 和中间元素 $a[mid]$ ，若 $x=a[mid]$ ，则 x 在 L 中的位置就是 mid ；如果 $x<a[mid]$ ，由于 a 是递增排序的，因此假如 x 在 a 中的话， x 必然排在 $a[mid]$ 的前面，所以只要在 $a[mid]$ 的前面查找 x 即可；如果 $x>a[i]$ ，同理只要在 $a[mid]$ 的后面查找 x 即可。无论是在前面还是后面查找 x ，其方法都和 a 中查找 x 一样，只不过是查找的规模缩小了。这就说明了此问题满足分治法的第二、三个适用条件



二分搜索技术

问题： 给定已按升序排好序的 n 个元素 $a[0:n-1]$ ，现在要在这 n 个元素中找出一特定元素 x 。

分析：

1. 该问题的规模缩小到一定的程度就可以容易地解决；
2. 该问题可以分解为若干个规模较小的相同问题；
3. 分解出的子问题的解可以合并为原问题的解；
4. 分解出的各个子问题是相互独立的。

很显然此问题分解出的子问题相互独立，即在 $a[i]$ 的前面或后面查找 x 是独立的子问题，因此满足分治法的第四个适用条件。

二分搜索算法

据此容易设计出二分搜索算法：

```
int BinarySearch(int A[], int target, int size) {
    int left = 0;
    int right = size - 1;
    int mid;
    while (left <= right) {
        mid = left + (right - left) / 2;
        if (A[mid] == target) {
            return mid;
        }
        if (A[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;}
```

算法复杂度分析：

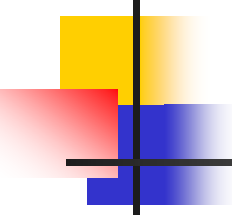
每执行一次算法中的while循环，待搜索数组的大小减少一半。因此，在最坏情况下，while循环被执行了 $O(\log_2 n)$ 次。循环体内运算需要 $O(1)$ 时间，因此整个算法在最坏情况下的计算时间复杂性为 $O(\log_2 n)$ 。

二分搜索算法(cont.)

- 二分搜索算法易于理解。
- 编写正确的二分搜索算法不易，**90%**人在**2个小时内**不能写出完全正确的二分算法。



第一个二分搜索算法在**1946年**提出，但是第一个完全正确的二分搜索算法却直到**1962年**才出现。



大整数的乘法

- 在复杂性计算时，都将加法和乘法运算当作基本运算处理，即加、乘法时间为常数。但上述假定仅在参加运算整数能在计算机表示范围内直接处理时才合理。

大整数的乘法(cont.)

问题：请设计一个有效的算法，可以进行两个 n 位大整数的乘法运算

◆小学的方法： $O(n^2)$ **✗ 效率低！！**

	n	$n \log_2 n$	n^2	n^3	1.5^n	2^n	$n!$
$n = 10$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	4 sec
$n = 30$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	18 min	10^{25} years
$n = 50$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	11 min	36 years	very long
$n = 100$	< 1 sec	< 1 sec	< 1 sec	1 sec	12,892 years	10^{17} years	very long
$n = 1,000$	< 1 sec	< 1 sec	1 sec	18 min	very long	very long	very long
$n = 10,000$	< 1 sec	< 1 sec	2 min	12 days	very long	very long	very long
$n = 100,000$	< 1 sec	2 sec	3 hours	32 years	very long	very long	very long
$n = 1,000,000$	1 sec	20 sec	12 days	31,710 years	very long	very long	very long

大整数的乘法(cont.)

问题：请设计一个有效的算法，可以进行两个 n 位大整数的乘法运算

◆小学的方法： $O(n^2)$ **✗ 效率低！！**

◆分治法：

假设： X 和 Y 都是 n 位二进制整数

$$X = \overset{n/2\text{位}}{\boxed{a}} \overset{n/2\text{位}}{\boxed{b}} \quad Y = \overset{n/2\text{位}}{\boxed{c}} \overset{n/2\text{位}}{\boxed{d}}$$

$$\begin{aligned} X &= a 2^{n/2} + b & Y &= c 2^{n/2} + d \\ \rightarrow XY &= ac 2^n + (ad+bc) 2^{n/2} + bd \end{aligned}$$